30 exercícios de AEDs 2 · nível intermediário

Complemento do repositório Estudos-Aeds-2 · Gabriel Cardoso Alvares de Oliveira

Esta lista vem depois dos 30 exercícios básicos. Cada exercício é um pouco mais completo: pede mais de uma operação, trata casos de erro e traz um exemplo para você conferir. Valem as convenções do repositório: `Celula`, `novaCelula`, `scanf` até o fim da entrada, `Erro` para casos inválidos, `MAX` e `ant`.

Resolva no papel primeiro, programe depois e confira nas demonstrações interativas. Em todo programa com `malloc`, libere a memória com `free`. Os quadros **CONFIRA** trazem resultados para você comparar com o seu.

Assunto	Exercícios	Pasta no Estudos-Aeds-2
Células	1 a 4	`CelulaEstrutura/`
Listas	5 a 11	`LIsta/`
Filas	12 a 18	`Fila/`
Pilha	19 a 22	`Pilha/`
Ordenações	23 a 30	`Ordenacoes/`

Células (`CelulaEstrutura/`)

1. Lista com cabeça: inserir no início.

Em `Celulacabeca.c`, leia números até o fim da entrada e insira cada um logo depois da célula cabeça. Imprima a lista e a quantidade `cont`. Depois explique por que essa inserção não precisa de `if (primeiro == NULL)`.

```
EXEMPLO
Entrada: 10 20 30
Saída:   30 20 10   (cont = 3)
```

2. Estatísticas da lista.

Em `Celula.c`, leia os números e imprima quantidade, soma, média (duas casas decimais), maior e menor valor. Se a lista ficar vazia, imprima `Erro`.

```
EXEMPLO
Entrada: 4 8 15 16
Saída:   quantidade 4, soma 43, média 10.75, maior 16, menor 4
```

3. Lista dupla nos dois sentidos.

Em `CelulaDupla.c`, insira os números no final e imprima a lista do primeiro ao último (com `prox`) e do último ao primeiro (com `ant`). No fim, libere todas as células com `free`.

```
EXEMPLO
Entrada: 1 2 3
Saída:   1 2 3
         3 2 1
```

4. Inverter sem criar células.

Em `Celula.c`, inverta a lista religando os ponteiros `prox`, sem chamar `novaCelula`. Imprima a lista antes e depois da inversão.

```
EXEMPLO
Entrada: 1 2 3 4
Saída:   1 2 3 4
         4 3 2 1
```

Listas (`LIsta/`)

5. Lista simples (vetor): inserir em posição.

Em `listaSimples.c`, guarde até 10 números no vetor, leia uma posição `pos` e um valor, e insira o valor em `pos`, deslocando os elementos para a direita. Imprima `Erro` se `pos` for inválida (menor que 0 ou maior que `n`) ou se o vetor estiver cheio.

```
EXEMPLO
Vetor: 5 8 2 9   pos = 2   valor = 7
Saída: 5 8 7 2 9
```

6. Lista flexível: remover do final com `realloc`.

Em `LIstaFlexivel.c` e `removerFinalListaFlexivel.c`, leia os números e remova elementos do final, um por vez, encolhendo o vetor com `realloc`. Depois de cada remoção, imprima o tamanho. Se a lista já estiver vazia, imprima `Erro`.

```
EXEMPLO
Vetor: 3 6 9   (quatro remoções)
Saída: 2, 1, 0 e depois Erro
```

7. Lista flexível: remover repetidos.

Leia números em um vetor flexível (que cresce com `realloc`) e imprima a lista sem repetidos, mantendo a primeira ocorrência de cada valor.

```
EXEMPLO
Entrada: 4 7 4 1 7 9
Saída:   4 7 1 9
```

8. Lista encadeada: inserir em posição.

Em `LIstaEncadeadaSimples.c`, insira um valor na posição `pos` (0 é o início), percorrendo a lista até a célula anterior. Imprima `Erro` se `pos` for maior que o tamanho da lista.

```
EXEMPLO
Lista: 10 20 30   valor = 25   pos = 2
Saída: 10 20 25 30
```

9. Lista encadeada: remover por valor.

Escreva duas funções: uma remove só a primeira ocorrência de um valor e a outra remove todas. Sem vazamento de memória. Trate a lista vazia (ao receber `Celula **primeiro`, teste `*primeiro == NULL`, e não `primeiro == NULL`).

```
EXEMPLO
Lista: 5 3 5 8 5   valor = 5
Primeira ocorrência: 3 5 8 5
Todas as ocorrências: 3 8
```

10. Lista encadeada: juntar duas listas ordenadas.

Monte duas listas ordenadas (pode ser direto no `main`) e construa uma terceira, também ordenada, sem ordenar depois: ande nas duas ao mesmo tempo, como na intercalação do Merge Sort.

EXEMPLO

```
A: 1 4 9    B: 2 4 7 8
Saída: 1 2 4 4 7 8 9
```

11. Lista dupla: remover por posição.

Tomando como base `ListaDuplaEncadeadaRemoverPrimeiro.c` e `ListaDuplaEncadeadaRemoverFInal.c`, escreva `removerPosicao`, que funciona no início, no meio e no final, mantendo `prox` e `ant` corretos. Depois de cada remoção, imprima a lista nos dois sentidos.

EXEMPLO

```
Lista: 1 2 3 4 5
remover pos 2 → 1 2 4 5
remover pos 0 → 2 4 5
remover pos 2 → 2 4   (de trás para frente: 4 2)
```

Filas (`Fila/`)

12. Fila encadeada: comandos.

Em `fila.c` e `filaRemover.c`, faça um programa que lê comandos: `I x` insere x, `R` remove e imprime o removido, `M` mostra a fila, `T` imprime o tamanho (mantido em um contador, sem percorrer a fila) e `F` encerra. Remover de fila vazia imprime `Erro`.

EXEMPLO

```
Comandos: I 5, I 8, M, R, T
Saídas:   5 8, 5, 1
```

13. Fila encadeada: intercalar duas filas.

Dadas duas filas A e B, gere uma terceira alternando um elemento de cada. Quando uma acabar, copie o resto da outra.

EXEMPLO

```
A: 1 2 3    B: 10 20
Saída: 1 10 2 20 3
```

14. Fila dupla: os dois lados.

Em `FilaInserirD.c` e `FilaRemoverD.c`, insira no final, remova do início e remova do final. Depois de cada operação, imprima a fila nos dois sentidos.

EXEMPLO

```
inserir 1 2 3 4
remover início → 2 3 4
remover final  → 2 3   (ao contrário: 3 2)
```

15. Fila dupla: palíndromo.

Insira uma sequência em uma fila dupla e diga se ela é palíndromo, comparando o início com o final e removendo os dois a cada passo. Imprima `sim` ou `nao`.

```
EXEMPLO
1 2 3 2 1 → sim
1 2 3 4   → nao
```

16. Fila circular: simulação.

Em `testefilacircular.c`, use `MAX = 4` (vetor de 5 posições). Execute: inserir 1, 2, 3, 4; remover duas vezes; inserir 5, 6, 7. Depois de cada passo, imprima `primeiro`, `ultimo` e o conteúdo da fila. Qual inserção resulta em `Erro: Fila cheia!`, e por quê?

```
CONFIRA
Ao final: primeiro = 2, ultimo = 1, fila = 3 4 5 6
```

17. Fila circular: tamanho e busca.

Escreva `tamanho(f)` usando só `primeiro`, `ultimo` e `MAX + 1`, sem laço. Escreva também `buscar(f, x)`, que devolve a posição lógica de x na fila (0 é o primeiro) ou -1 se não existir. Confira `tamanho` no estado final do exercício 16.

```
CONFIRA
primeiro = 2, ultimo = 1, MAX = 4 → tamanho = 4
buscar(f, 5) → 2        buscar(f, 9) → -1
```

18. Fila circular: versão com contador.

Reescreva a fila circular com um vetor de exatamente `MAX` posições e um campo `qtd`, sem a posição de segurança. Compare com a versão do repositório: como ficam os testes de fila cheia e fila vazia? Quando cada versão é melhor? Responda em poucas linhas.

Pilha (`Pilha/`)

19. Pilha: calculadora pós-fixa.

Com base em `PilhaInserir.c` e `PilhaRemover.c`, avalie uma expressão pós-fixa com números de um dígito e os operadores `+ - * /`. Empilhe os números e, a cada operador, desempilhe dois operandos e empilhe o resultado. Se faltar operando, imprima `Erro`.

```
EXEMPLO
Entrada: 3 4 + 2 *
Saída:   14
```

20. Pilha: parênteses balanceados.

Leia uma expressão e diga se `()`, `[]` e `{}` estão balanceados. Imprima `sim` ou `nao`.

```
EXEMPLO
{[()]} → sim
([)]   → nao
((     → nao
```

21. Pilha: decimal para binário.

Leia um inteiro positivo e imprima seu binário: empilhe os restos das divisões por 2 e desempilhe para imprimir.

EXEMPLO

```
Entrada: 13
Saída:   1101
```

22. Pilha: fila com duas pilhas.

Implemente `inserir` e `remover` de uma fila usando apenas duas pilhas. Explique quantas operações cada uma custa no pior caso.

EXEMPLO

```
inserir 1, 2, 3 → remover = 1
inserir 4       → remover = 2
```

Ordenações (`Ordenacoes/`)

23. Insertion Sort: contar comparações.

Em `insercao.c`, conte quantas vezes a condição `arr[j] > key` é avaliada. Rode nos dois vetores abaixo e explique por que o segundo caso custa tão menos.

CONFIRA

```
5 2 4 6 1 3 → 12 comparações
1 2 3 4 5 6 →  5 comparações
```

24. Selection Sort: acompanhar cada passo.

Em `selecao.c`, imprima o vetor ao final de cada passo do laço externo e conte as trocas, sem trocar quando o menor já está na posição certa. Compare com o Insertion Sort: o que acontece com cada um quando o vetor já vem ordenado?

CONFIRA

```
Vetor: 64 25 12 22 11
Após cada passo:
11 25 12 22 64
11 12 25 22 64
11 12 22 25 64
11 12 22 25 64   (3 trocas no total)
```

25. Counting Sort: contagem e acumulado.

Em `countingsort.c`, leia notas inteiras de 0 a 10. Imprima o vetor `count`, o vetor acumulado (soma de prefixos) e o vetor ordenado. Explique por que o laço final anda de trás para frente (estabilidade).

EXEMPLO

```
Notas: 3 1 3 0 2 1
count (0..3):   1 2 1 2
acumulado:      1 3 4 6
ordenado:       0 1 1 2 3 3
```

26. Radix Sort: passada por passada.

Em `Radixsort.c`, imprima o vetor depois de cada passada (unidades, dezenas, centenas). Por que cada passada precisa ser estável?

CONFIRA

```
Entrada: 170 45 75 90 802 24 2 66
unidades: 170 90 802 2 24 45 75 66
dezenas:  802 2 24 45 66 170 75 90
centenas: 2 24 45 66 75 90 170 802
```

27. Bucket Sort: em qual balde cai cada valor?

Em `Bucketsort.c` (`NUM_BUCKETS = 5`, `idx = (v - min) * 5 / range`), calcule à mão o balde de cada valor do vetor abaixo, ordene cada balde e junte tudo. Depois execute o programa e compare.

CONFIRA

```
Vetor: 29 25 3 49 9 37 21 43   (min = 3, max = 49)
Baldes: 29→2  25→2  3→0  49→4  9→0  37→3  21→1  43→4
Resultado: 3 9 21 25 29 37 43 49
```

28. Merge Sort: contar inversões.

Em `mergesort.c`, some à intercalação um contador de inversões (pares fora de ordem): quando um elemento da metade direita passa na frente de elementos da esquerda, some quantos restavam na esquerda. Imprima o total ao final.

CONFIRA

```
2 4 1 3 5 → 3 inversões   (2,1) (4,1) (4,3)
38 27 43 3 9 82 10 → 11 inversões
```

29. Quick Sort: partição passo a passo.

Em `Quicksort.c` (partição de Lomuto, pivô no último elemento), imprima o vetor depois de cada chamada de `partition`, junto com a posição final do pivô. Em seguida, ordene um vetor já ordenado e explique por que esse é o pior caso.

CONFIRA

```
Vetor: 3 6 1 5 2 4
Após a 1ª partição: 3 1 2 4 6 5   (pivô na posição 3)
```

30. Heap Sort: construir o heap e extrair.

Em `Heapsort.c`, imprima o vetor depois de construir o heap máximo e depois de cada extração. Informe os índices do pai e dos filhos de um elemento `i` e desenhe a árvore do vetor.

CONFIRA

```
Vetor: 4 10 3 5 1
Heap máximo: 10 5 3 4 1
```